

PgBouncer — PostgreSQL Connection Pooling

Table of Contents

1. [Learning Objectives](#)
 2. [Why Connection Pooling](#)
 3. [PgBouncer Architecture](#)
 4. [Pooling Modes](#)
 5. [Installation](#)
 6. [pgbouncer.ini Configuration](#)
 7. [userlist.txt — Authentication](#)
 8. [Monitoring and Admin Console](#)
 9. [PgBouncer with Patroni/HA](#)
 10. [TLS/SSL Configuration](#)
 11. [Common Mistakes](#)
 12. [Best Practices](#)
 13. [Performance Considerations](#)
 14. [Interview Questions](#)
 15. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Choose the right pooling mode for your workload
 - Configure a production-grade pgbouncer.ini
 - Monitor connection pool health and performance
 - Integrate PgBouncer with a Patroni HA cluster
 - Troubleshoot connection pool exhaustion
 - Implement secure connections through PgBouncer
-

Why Connection Pooling

PostgreSQL is a process-per-connection database — each client connection forks a new backend process consuming ~5-10MB of shared memory and causing context switching overhead.

WITHOUT PGBOUNCER:

1000 clients → 1000 PostgreSQL backends → ~5GB RAM consumed by idle processes

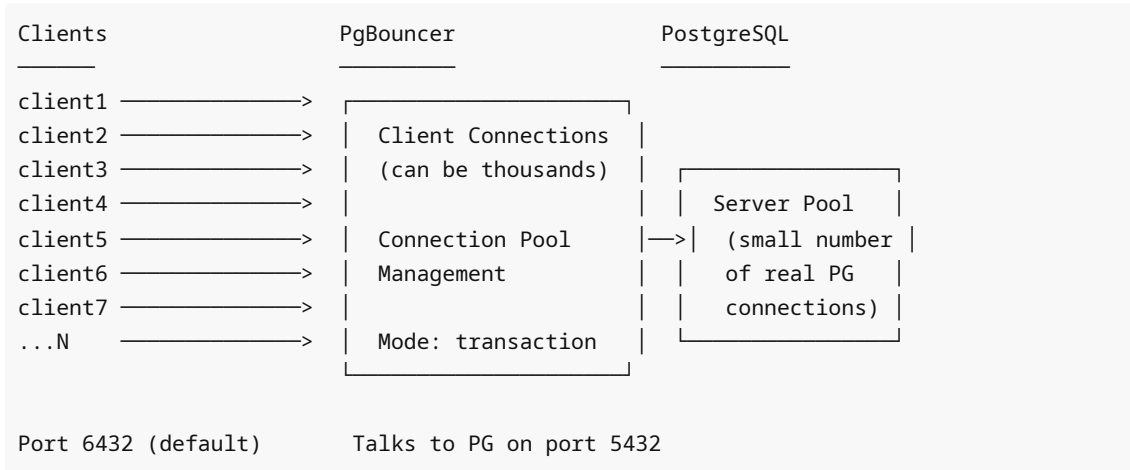
WITH PGBOUNCER:

1000 clients → PgBouncer → 20 PostgreSQL backends → ~100MB RAM
(multiplexes)

Problem	Without Pooling	With PgBouncer
1000 idle web workers	1000 PG backends	20 PG backends
Connection storms	Server may OOM	PgBouncer queues
Short-lived connections	Connection overhead per request	Persistent pool reused

Memory usage	High	Low
--------------	------	-----

PgBouncer Architecture



Pool Types

PgBouncer maintains a **server pool** for each (database, user) pair.

```
Pool for (mydb, appuser):
  server connections: 20 (to PostgreSQL)
  client connections: up to max_client_conn waiting or connected
```

Pooling Modes

Session Mode (least multiplexing)

Client gets a dedicated server connection for the ENTIRE session duration.
Released when client disconnects.

```
Timeline:
client1 —connect—> [server conn assigned]
client1 —query1—>
client1 —idle—> (server conn still assigned, not usable by others)
client1 —query2—>
client1 —disconnect—> [server conn returned to pool]
```

Use case: Applications that use session-level features:

- SET LOCAL statements that persist
- LISTEN/NOTIFY
- Advisory locks
- Temporary tables
- Prepared statements (session-level)

Performance: Least efficient – like not having a pool for short sessions

Transaction Mode (recommended for most apps)

Client gets a server connection only for the duration of a TRANSACTION.
Released back to pool immediately after COMMIT/ROLLBACK.

Timeline:

```
client1 —BEGIN——> [server conn assigned from pool]
client1 —INSERT——>
client1 —COMMIT——> [server conn returned to pool]
client1 —idle——> (no server conn held)
client2 —BEGIN——> [gets same server conn from pool]
```

Use case:

- Standard web applications (Django, Rails, FastAPI)
- REST APIs with short transactions
- Any autocommit workload

Restrictions:

- Session-level SET statements don't persist
- LISTEN/NOTIFY not supported (session-level)
- Named prepared statements don't work across transactions
- Advisory locks: session-level ones behave like transaction-level

Performance: Best – N clients need only $M \ll N$ server connections

Statement Mode (most aggressive)

Server connection returned after EACH STATEMENT (even mid-transaction).
This means multi-statement transactions are broken!

Use case: Only for fully autocommit, single-statement workloads
Very rarely appropriate.

Restriction: Cannot use multi-statement transactions AT ALL.

Mode Comparison

Feature	session	transaction	statement
Multi-statement txn	Yes	Yes	No
SET session vars persist	Yes	No*	No
Temp tables	Yes	No	No
LISTEN/NOTIFY	Yes	No	No
Prepared statements	Yes	No**	No
Advisory locks (session)	Yes	No	No
Multiplexing efficiency	Low	High	Highest

Installation

```
# Ubuntu/Debian
sudo apt-get install -y pgbouncer

# RHEL/CentOS
sudo yum install -y pgbouncer

# From source
wget https://www.pgbouncer.org/downloads/files/1.23.0/pgbouncer-1.23.0.tar.gz
tar xf pgbouncer-1.23.0.tar.gz
cd pgbouncer-1.23.0
./configure --prefix=/usr --with-openssl
make && sudo make install

# Verify
pgbouncer --version
```

pgbouncer.ini Configuration

```
; /etc/pgbouncer/pgbouncer.ini
; Complete production configuration

;-----
; DATABASE DEFINITIONS
;-----
[databases]

; Syntax: <pool_name> = host=<pg_host> port=<pg_port> dbname=<pg_dbname> [options]

; Simple database mapping (same name on PG)
mydb = host=127.0.0.1 port=5432 dbname=mydb

; Rename database (clients connect to "app", PG has "production_db")
app = host=192.168.1.100 port=5432 dbname=production_db

; Specify user (all pool connections use this PG user)
; Clients still authenticate with their own user
readonly_pool = host=192.168.1.101 port=5432 dbname=mydb user=readonly_user

; Pool size override for specific database
highload_db = host=192.168.1.100 port=5432 dbname=highload pool_size=50

; Read replica pool
mydb_replica = host=192.168.1.101 port=5432 dbname=mydb

; Catch-all: any database name connects to same PG server
; * = host=192.168.1.100 port=5432

; Force authentication user
; mydb = host=192.168.1.100 port=5432 dbname=mydb auth_user=pgbouncer_auth
```

```

;-----
; PGBOUNCER SETTINGS
;-----

[pgbouncer]

;— Listen address
listen_addr = 0.0.0.0          ; Listen on all interfaces (or specific IP)
listen_port = 6432             ; PgBouncer port (app connects here)

;— Authentication
auth_type = scram-sha-256      ; or md5, plain, hba, cert
                                ; 'hba' uses pg_hba.conf-style auth
auth_file = /etc/pgbouncer/userlist.txt ; User password file

; Optional: Use PostgreSQL's HBA for auth
; auth_hba_file = /etc/pgbouncer/pg_hba.conf

; Auth query: look up passwords from PostgreSQL itself
; auth_user = pgbouncer_auth
; auth_query = SELECT username, passwd FROM pg_shadow WHERE username=$1

;— Pooling mode
pool_mode = transaction        ; session | transaction | statement

;— Pool size limits
max_client_conn = 2000         ; Maximum total client connections to PgBouncer
default_pool_size = 25         ; Default server connections per (db, user) pool
min_pool_size = 5              ; Minimum server connections to keep alive
reserve_pool_size = 5          ; Extra connections available when pool exhausted
reserve_pool_timeout = 5       ; Seconds to wait for reserve pool before error

; max_db_connections = 0        ; Limit server connections per database (0=unlimited)
; max_user_connections = 0      ; Limit server connections per user (0=unlimited)

;— Timeouts
server_idle_timeout = 600      ; Remove idle server connections after 10 minutes
client_idle_timeout = 0        ; Remove idle client connections (0=disabled)
server_connect_timeout = 15     ; Timeout for new PG connections
server_login_retry = 15        ; Retry failed PG connection after N seconds
query_timeout = 0              ; Cancel queries running longer than N seconds
                                (0=disabled)
query_wait_timeout = 120       ; Client waits max 2 minutes for free server conn
client_login_timeout = 60      ; Time client has to log in to PgBouncer
idle_transaction_timeout = 0    ; Disconnect idle-in-transaction clients (0=disabled)
                                ; Set to 60 to detect stuck transactions

;— Connection behavior
server_reset_query = DISCARD ALL ; Run after each transaction (in transaction
mode)
                                ; Clears session state (SET vars, temp tables,
etc.)

```

```

; server_reset_query_always = 0      ; Only run reset if session was dirty

server_check_query = select 1      ; Verify server connection is alive
server_check_delay = 30            ; Check every 30 seconds

;— TLS (see TLS section)
; server_tls_sslmode = require
; server_tls_ca_file = /etc/ssl/certs/ca-certificates.crt
; client_tls_sslmode = require
; client_tls_key_file = /etc/pgbouncer/pgbouncer.key
; client_tls_cert_file = /etc/pgbouncer/pgbouncer.crt

;— Logging
logfile = /var/log/pgbouncer/pgbouncer.log
pidfile = /var/run/pgbouncer/pgbouncer.pid
log_connections = 1                ; Log client connects
log_disconnections = 1             ; Log client disconnects
log_pooler_errors = 1              ; Log pool errors
log_stats = 1                      ; Log pool statistics every stats_period seconds
stats_period = 60                  ; Statistics log interval (seconds)
verbose = 0                        ; Set to 1 for debug (very chatty)

;— Admin
admin_users = postgres             ; Users allowed to connect to pgbouncer admin DB
stats_users = monitoring           ; Users allowed to view stats (SELECT only)

;— Operating system
unix_socket_dir = /var/run/postgresql ; UNIX socket directory
unix_socket_mode = 0777

;— Per-database application name
; application_name_add_host = 1     ; Add host info to application_name

```

userlist.txt — Authentication

```

# /etc/pgbouncer/userlist.txt
# Format: "username" "password_hash"
# Generate hash: echo -n "password" | md5sum
# Or for scram-sha-256: use pg_shadow table values

# Method 1: MD5 hash (format: md5 + hash(password + username))
"appuser" "md5a4e0f86d421b8e3e1af6c8de65ac3e2"

# Method 2: Plain text (not recommended for production)
"appuser" "password123"

# Method 3: Use PostgreSQL auth_query (recommended)
# Let PgBouncer query PG for passwords directly
# Remove userlist.txt, set auth_user and auth_query in pgbouncer.ini

```

```
# Method 4: SCRAM hash (copy from pg_shadow)
"appuser" "SCRAM-SHA-256$4096:..."

# Generate via SQL:
# SELECT passwd FROM pg_shadow WHERE username = 'appuser';
# Copy the value into userlist.txt with quotes
```

Using auth_query (Recommended)

```
-- On PostgreSQL: create auth user with minimal privileges
CREATE ROLE pgbouncer_auth WITH LOGIN PASSWORD 'auth_password';
GRANT SELECT ON pg_shadow TO pgbouncer_auth;
```

```
; pgbouncer.ini
auth_user = pgbouncer_auth
auth_query = SELECT username, passwd FROM pg_shadow WHERE username=$1
; Remove or empty the auth_file when using auth_query
```

Monitoring and Admin Console

```
# Connect to PgBouncer admin console
psql -h 127.0.0.1 -p 6432 -U postgres pgbouncer
# Database name must be "pgbouncer"
```

```
-- Show all configured databases and their pool stats
SHOW DATABASES;
-- name | host | port | database | force_user | pool_size | reserve_pool | pool_mode
| max_connections

-- Show all pools (per database+user combination)
SHOW POOLS;
-- database | user | cl_active | cl_waiting | sv_active | sv_idle | sv_used |
sv_tested | sv_login | maxwait | pool_mode

-- Column explanation:
-- cl_active: Client connections currently executing a query
-- cl_waiting: Client connections waiting for a free server connection
-- sv_active: Server connections currently used by a client
-- sv_idle: Server connections idle, ready to use
-- sv_used: Server connections returned but not yet idle (being reset)
-- maxwait: Maximum seconds a client has been waiting (alert if > 0!)

-- Show client connections
SHOW CLIENTS;

-- Show server connections
SHOW SERVERS;
```

```
-- Show statistics
SHOW STATS;
-- total_query_count, total_xact_count, avg_query_time, avg_xact_time

-- Show statistics per second
SHOW STATS_AVERAGES;

-- Configuration
SHOW CONFIG;

-- Current pool sizes
SHOW LISTS;

-- Memory usage
SHOW MEM;
```

Operational Commands

```
-- In pgbouncer admin console:

-- Reload configuration (without restart)
RELOAD;

-- Pause all connections (for maintenance)
PAUSE;

-- Resume after pause
RESUME;

-- Kill a specific client connection
KILL client_id;

-- Disconnect all server connections for a database
RECONNECT mydb;

-- Show version
SHOW VERSION;
```

Monitoring Script

```
#!/bin/bash
# pgbouncer_monitor.sh
PGBOUNCER_HOST=127.0.0.1
PGBOUNCER_PORT=6432
PGBOUNCER_USER=postgres

psql -h $PGBOUNCER_HOST -p $PGBOUNCER_PORT -U $PGBOUNCER_USER pgbouncer -c "
SELECT
```



```

        database,
        user,
        cl_active,
        cl_waiting,
        sv_active,
        sv_idle,
        maxwait
FROM pools
ORDER BY cl_waiting DESC, database;
"

# Alert if clients are waiting
WAITING=$(psql -h $PGBOUNCER_HOST -p $PGBOUNCER_PORT -U $PGBOUNCER_USER pgbouncer -t -c \
    "SELECT SUM(cl_waiting) FROM pools;")
if [ "$WAITING" -gt "0" ]; then
    echo "WARNING: $WAITING clients are waiting for connections!"
fi

```

PgBouncer with Patroni/HA

When used with Patroni, PgBouncer must be updated when the primary changes.

Strategy 1: PgBouncer per Node (Recommended)

```

App →> HAProxy →> PgBouncer1 →> PG Node1 (primary)
                        →> PgBouncer2 →> PG Node2 (replica)
                        →> PgBouncer3 →> PG Node3 (replica)

```

HAProxy health checks /primary and /replica Patroni endpoints.
 Each PgBouncer talks to its local PostgreSQL.
 On failover: HAProxy reroutes, PgBouncers reconnect automatically.

Strategy 2: Centralized PgBouncer with PAUSE/RESUME

```

#!/bin/bash
# patroni_callback.sh
# Configure in patroni.yml as on_role_change callback

ACTION=$1
ROLE=$2
CLUSTER=$3

PGBOUNCER_HOST=pgbouncer.internal
PGBOUNCER_PORT=6432
PGBOUNCER_ADMIN=postgres

case "$ACTION" in
    on_start|on_restart)
        if [ "$ROLE" == "master" ]; then

```

```

        # New primary: resume PgBouncer
        psql -h $PGBOUNCER_HOST -p $PGBOUNCER_PORT -U $PGBOUNCER_ADMIN pgbouncer \
        -c "RESUME;"
        echo "PgBouncer resumed for new primary"
    fi
    ;;
on_stop)
    # Stopping: pause connections gracefully
    psql -h $PGBOUNCER_HOST -p $PGBOUNCER_PORT -U $PGBOUNCER_ADMIN pgbouncer \
    -c "PAUSE;"
    ;;
on_reload)
    # After config reload, reconnect PgBouncer to new primary
    psql -h $PGBOUNCER_HOST -p $PGBOUNCER_PORT -U $PGBOUNCER_ADMIN pgbouncer \
    -c "RECONNECT;"
    ;;
esac

```

```

# patroni.yml
postgresql:
  callbacks:
    on_role_change: /etc/patroni/patroni_callback.sh
    on_start: /etc/patroni/patroni_callback.sh
    on_stop: /etc/patroni/patroni_callback.sh

```

TLS/SSL Configuration

```

; pgbouncer.ini – TLS settings

; Connections FROM clients TO PgBouncer
client_tls_sslmode = require
client_tls_key_file = /etc/pgbouncer/server.key
client_tls_cert_file = /etc/pgbouncer/server.crt
client_tls_ca_file = /etc/ssl/certs/ca-certificates.crt
client_tls_protocols = secure ; TLSv1.2+

; Connections FROM PgBouncer TO PostgreSQL
server_tls_sslmode = require
server_tls_ca_file = /etc/ssl/certs/ca-certificates.crt
server_tls_protocols = secure

```

Common Mistakes

1. **Using session mode for short-lived connections** — no multiplexing benefit
2. **Not setting `idle_transaction_timeout`** — connections stuck in idle-in-transaction
3. **Pool size too small** — clients queue and time out under load
4. **auth_type mismatch** — PgBouncer `auth_type` must match what PostgreSQL expects

5. **Not monitoring `maxwait`** — clients silently wait and eventually time out
 6. **Using prepared statements with transaction mode** — they break
 7. **Not using `DISCARD ALL` as `server_reset_query`** — session state leaks between clients
 8. **PgBouncer as a single point of failure** — should have multiple PgBouncer instances
 9. **Connecting to `pgbouncer` db for apps** — admin console, not app data
 10. **Not handling Patroni failover** — PgBouncer keeps connections to old primary
-

Best Practices

1. Use **transaction mode** for stateless web applications
 2. Set **`idle_transaction_timeout = 60`** to catch stuck transactions
 3. Deploy **multiple PgBouncer instances** behind a load balancer
 4. Use **`auth_query`** instead of hardcoded `userlist.txt`
 5. Monitor **`cl_waiting` and `maxwait`** — spikes indicate pool exhaustion
 6. Set **`pool_size`** based on PostgreSQL `max_connections` / number of pools
 7. Use **`server_reset_query = DISCARD ALL`** in transaction mode
 8. Enable **TLS** for client connections in production
 9. Keep **`log_connections = 1`** for auditing connection patterns
 10. Test **failover** with PgBouncer in the path to verify reconnect behavior
-

Performance Considerations

```
; Tuning for high-throughput OLTP
default_pool_size = 25           ; Start here: ~25 per database per user
max_client_conn = 5000          ; Handle large numbers of clients
reserve_pool_size = 10          ; Burst capacity
reserve_pool_timeout = 1         ; Give only 1 second before error (fast-fail)
server_idle_timeout = 300        ; Keep idle connections for 5 minutes
query_wait_timeout = 30          ; Fail fast if pool is full (30s)
```

Interview Questions

Q1: What is the difference between session, transaction, and statement pooling modes?

A: Session mode assigns a server connection per client session (released on disconnect). Transaction mode assigns a server connection per transaction (released after COMMIT/ROLLBACK). Statement mode releases after each statement. Transaction mode provides the best multiplexing for typical web apps, but breaks session-level features like temp tables, LISTEN/NOTIFY, and named prepared statements.

Q2: Why is PgBouncer necessary for PostgreSQL?

A: PostgreSQL creates a separate OS process per connection (~5-10MB RAM each). With 500+ concurrent connections, this wastes memory and causes excessive context switching. PgBouncer maintains a small pool of real PostgreSQL connections (e.g., 25) while allowing thousands of client connections, multiplexing them efficiently.

Q3: What is `server_reset_query` and why is it important?

A: In transaction mode, PgBouncer reuses server connections across multiple clients. If a client sets session state (SET vars, opened temp tables), the next client using that connection would inherit it.

`server_reset_query = DISCARD ALL` cleans all session state before returning the connection to the pool, preventing data leakage between clients.

Q4: How do you handle prepared statements with PgBouncer?

A: Named prepared statements are session-level in PostgreSQL and don't work with transaction mode pooling (the statement exists on a specific server connection that may be reassigned). Solutions: (1) Use simple query protocol (no named prepared statements). (2) Use session mode pooling. (3) Use protocol-level prepared statement passthroughs (pgbouncer 1.21+ has some support). (4) Use `PREPARE / EXECUTE` with unique statement names per transaction.

Q5: What does `cl_waiting` in `SHOW POOLS` indicate?

A: The number of client connections waiting for a free server connection from the pool. Non-zero `cl_waiting` means the pool is exhausted and clients are queuing. Persistent `cl_waiting` requires increasing `default_pool_size` or reducing connection demand.

Q6: How do you reload PgBouncer configuration without restart?

A: Connect to the admin console (`psql -p 6432 pgbouncer`) and run `RELOAD;` . This re-reads `pgbouncer.ini` and applies non-restart-requiring changes (pool sizes, timeouts, log settings). Some changes (`listen_addr`, `listen_port`) require full restart.

Q7: What happens to existing connections during PgBouncer reload?

A: Active connections continue unaffected. Idle server connections may be reconnected if server address changed. `RELOAD` applies most configuration changes live. For major changes (like changing the backend PostgreSQL server), use `RECONNECT <dbname>` to force reconnection.

Q8: How does PgBouncer integrate with Patroni for automatic failover?

A: Multiple approaches: (1) Per-node PgBouncer (each talks to local PostgreSQL; HAProxy handles routing to the primary node). (2) Patroni callbacks (`on_role_change` script that pauses/resumes/reconnects PgBouncer). (3) Consul service + PgBouncer watching service health. The key requirement is that PgBouncer's server connection is redirected to the new primary after failover.

Exercises and Solutions

Exercise 1: Design Connection Pool for Microservices

```
; 10 microservices, each with up to 50 app workers
; PostgreSQL max_connections = 200
; Design: pool_size = 200 / 10 services = 20 per service
```

[databases]

```
service_auth      = host=127.0.0.1 port=5432 dbname=auth_db      pool_size=20
service_orders    = host=127.0.0.1 port=5432 dbname=orders_db    pool_size=20
service_users     = host=127.0.0.1 port=5432 dbname=users_db     pool_size=20
service_payments  = host=127.0.0.1 port=5432 dbname=payments_db  pool_size=20
```

[pgbouncer]

```
pool_mode = transaction
```

```
max_client_conn = 1000 ; 10 services * 50 workers * 2 = comfortable headroom
default_pool_size = 20
```

Exercise 2: Monitor Pool Exhaustion

```
#!/bin/bash
# Alert when pool is being exhausted
psql -h 127.0.0.1 -p 6432 -U postgres pgbouncer -t -c \
"SELECT database, user, cl_waiting, maxwait
FROM pools
WHERE cl_waiting > 0 OR maxwait > 5
ORDER BY maxwait DESC;" | while read line; do
    echo "POOL ALERT: $line"
done
```

Cross-References

- [06_patroni.md](#) — Patroni integration
- [08_haproxy_load_balancing.md](#) — HAProxy in front of PgBouncer
- [09_ha_architecture_patterns.md](#) — Complete HA with PgBouncer